

Estudio de la complejidad temporal de algoritmos iterativos de búsqueda y ordenamiento

Andrés Barbosa[†], Milton Hernández[†] y Iván Gallardo[†]

[†]Universidad de Magallanes

Este informe fue compilado el 14 de abril de 2026

Resumen

En este informe se presenta un estudio comparativo de la complejidad temporal de algoritmos iterativos de búsqueda y ordenamiento aplicados a una base de datos ficticia de deportistas. Los experimentos se llevaron a cabo sobre conjuntos de datos de hasta 7000 elementos en tres escenarios de distribución distintos (Elementos previamente ordenados, en orden inverso y en orden aleatorio). Los resultados demostraron una correspondencia significativa entre el análisis teórico de complejidad temporal y el comportamiento experimental observado.

Keywords: Algoritmos iterativos, Complejidad temporal, Ordenamiento, Búsqueda, Análisis experimental.

■ Índice

1	Introducción	3
2	Objetivo principal	3
2.1	Objetivos secundarios	3
3	Desarrollo	4
3.1	Análisis teórico de complejidad	4
	Búsqueda secuencial (Linear Search) • Búsqueda Binaria Iterativa (Binary Search) • Ordenamiento por intercambio (Swap sort) • Ordenamiento por Inserción (Insertion sort) • Ordenamiento por selección (Selection sort) • Ordenamiento de coctelera (Cocktail-Shaker Sort)	
3.2	Implementación de Algoritmos de Ordenamiento	10
3.3	Generador de deportistas	10
3.4	Verificación Experimental de la Complejidad Temporal	11
4	Resultados experimentados	12
4.1	Algoritmos de Búsqueda	12
4.2	Algoritmos de Ordenamiento	13
	Arreglo Ordenado (Mejor Caso) • Arreglo en Orden Inverso (Peor Caso) • Arreglo Desordenado (Caso Promedio) • Comportamiento de Cocktail Shaker Sort:	

5	Conclusiones	15
6	Contribución por integrante	16
6.1	Andrés Barbosa	16
6.2	Milton Hernández	16
6.3	Iván Gallardo	16
7	anexos	17

1. Introducción

El diseño de algoritmos tiene como objetivo encontrar soluciones a problemas de manera eficiente teniendo en cuenta las limitaciones de tiempo y memoria que existen dentro de los sistemas informáticos.

Dos de los problemas más comunes en el día a día de la computación son el **ordenamiento** de datos y la **búsqueda** de información dentro de estructuras de datos; ambos problemas se pueden afrontar desde distintas miradas: algoritmos iterativos, recursivos, uso de estructuras de datos especiales, etc.

El presente informe aborda un problema de ordenamiento y búsqueda dentro de una base de datos ficticia de deportistas (generada proceduralmente) con el fin de exponer algunas de las soluciones más conocidas a estos problemas (*Swap sort*, *Insertion sort*, *Selection sort*, *Cocktail shaker sort*, *búsqueda secuencial* y *búsqueda binaria*), analizando su comportamiento a nivel teórico y experimental.

2. Objetivo principal

Analizar y comparar el comportamiento de algunos algoritmos de ordenamiento y búsqueda estableciendo una correspondencia entre los resultados teóricos y experimentales.

2.1. Objetivos secundarios

1. Comprender el funcionamiento y la implementación de los algoritmos de ordenamiento y búsqueda utilizados.
2. Implementar un sistema sobre el cual aplicar y evaluar el desempeño de los algoritmos.
3. Documentar y analizar los resultados obtenidos experimentalmente, contrastándolos con el análisis teórico mediante gráficos comparativos que permitan extraer conclusiones sobre el comportamiento de cada algoritmo.

3. Desarrollo

3.1. Análisis teórico de complejidad

A continuación se presentan los algoritmos implementados y el consiguiente estudio de su complejidad temporal mediante el uso del orden asintótico.

3.1.1. Búsqueda secuencial (Linear Search)

Este algoritmo consiste en recorrer el arreglo por medio de sus índices $[0..n - 1]$ y en el momento en que se encuentre el elemento deseado se retorna su índice.

La implementación del algoritmo que a continuación se detalla tiene una variable respuesta r con un valor inicial de n , recorre el arreglo en busca del elemento x solicitado y de encontrarlo retorna su índice; si no se encuentra retornará n como resultado.

El pseudocódigo asociado a dicho algoritmo es el siguiente:

Algorithm 1 Búsqueda secuencial #1 $\triangleright V[0..n - 1]$

```

1: procedure BSEC( $V, x$ )
2:    $r \leftarrow n$ 
3:    $i \leftarrow 0$ 
4:   while  $i \leq n \wedge r = n$  do
5:     if  $x = V[i]$  then
6:        $r \leftarrow i$ 
7:      $i \leftarrow i + 1$ 
8:   return  $r$ 

```

Se puede realizar el siguiente análisis de la ejecución del algoritmo.

Línea	Costo	Repeticiones en peor caso
2, 3	c_1	1
4	c_2	n
5	c_3	n
6	c_4	0
7	c_5	n
8	c_6	1

El **peor caso** se da cuando el elemento no está en el arreglo, por tanto todas las líneas se ejecutan la máxima cantidad de veces:

$$T(n) = c_1 + (c_2 + c_5)n + c_8 \in O(n)$$

El **mejor caso** ocurre cuando el elemento está en la primera posición del arreglo, encontrando el elemento en el menor tiempo posible:

$$T(n) = c_1 + c_2 + c_3 + c_4 + c_5 + c_6 \in O(1)$$

Finalmente el **caso promedio** ocurre cuando el elemento está en el medio del arreglo, con lo que la complejidad continúa escalando de manera lineal dependiendo del tamaño del mismo, manteniendo un orden de $O(n)$.

3.1.2. Búsqueda Binaria Iterativa (Binary Search)

Este algoritmo se basa en la idea de reducir el rango de búsqueda a la mitad en cada paso.

En el anterior algoritmo (3.1.1) se buscaba desde el inicio hasta el final, de modo que si el elemento x se encontraba en la posición 1500 se tendrían que recorrer 1500 elementos para encontrarlo.

La búsqueda binaria (partiendo de la premisa que el arreglo se encuentra **previamente ordenado**) parte la búsqueda desde la mitad del arreglo comparando el valor presente en esa posición ($V[m]$) con el valor buscado (x), si el valor medio es mayor que el buscado entonces se busca en la mitad izquierda del arreglo (donde están los elementos menores que $V[m]$), en caso contrario se busca en la mitad derecha del arreglo (donde están los elementos mayores que m). Tomada esa decisión se repiten los pasos anteriores hasta descartar todo el arreglo o encontrar el elemento deseado.

El pseudocódigo asociado a dicho algoritmo es el siguiente:

Algorithm 2 Búsqueda Binaria Iterativa $\triangleright V[0..n - 1]$

```

1: procedure BINARYSEARCH( $V, beg, end, x$ )
2:   while  $beg \leq end$  do
3:      $m \leftarrow beg + \frac{end - beg}{2}$ 
4:     if  $V[m] = x$  then
5:       return  $m$ 
6:     else if  $V[m] < x$  then
7:        $beg \leftarrow m + 1$ 
8:     else
9:        $end \leftarrow m - 1$ 
10:  return -1

```

Para analizar la complejidad temporal de este algoritmo resulta necesario responder una pregunta clave: *¿Cuántas veces como máximo se ejecutará la línea 2 y, por consiguiente, el bucle While?* El bucle While, en cada iteración, **divide** el problema a la mitad; por lo tanto, la respuesta puede encontrarse determinando cuántas veces debe duplicarse el caso base para alcanzar el tamaño original del problema (n), lo cual equivale a $\log_2(n)$ veces.

El **peor caso** se da cuando el elemento no está en el arreglo, por tanto todas las líneas se ejecutan la máxima cantidad de veces; el bucle While se ejecuta $\log_2(n)$ y todas sus operaciones internas son de tiempo constante por tanto la complejidad será:

$$T(n) \in O(\log_2(n)) \equiv O(\log(n))$$

El **mejor caso** ocurre cuando el elemento está justo en la primera posición m calculada, encontrando el elemento en el menor tiempo constante:

$$T(n) \in O(1)$$

Finalmente el **caso promedio** ocurre cuando el elemento está en el medio del arreglo, con lo que la complejidad continúa escalando de manera logarítmica dependiendo del tamaño del mismo, manteniendo un orden de $O(\log(n))$.

3.1.3. Ordenamiento por intercambio (Swap sort)

Conocido popularmente como “BubbleSort” es un método sencillo que consiste en revisar cada elemento del conjunto con el siguiente, intercambiándolos de posición si están en orden inverso. Es necesario revisar varias veces toda la lista hasta que ya no se necesiten intercambios.

En su implementación más básica el algoritmo recorre el arreglo n^2 veces en cualquier caso (esté el arreglo ordenado, desordenado parcialmente o en orden inverso); sin embargo en su implementación más eficiente el programa levanta una bandera en caso de haber realizado un intercambio en uno de los recorridos del arreglo lo que permite que ante la no elevación de la misma el algoritmo detecte un arreglo completamente ordenado terminando mucho antes su ejecución.

El Pseudocódigo asociado a dicho algoritmo es el siguiente:

Algorithm 3 Ordenamiento por intercambio ▷ $V[0..n - 1]$

```

1: procedure SWAPSORT( $V$ )
2:   for  $i \leftarrow 1$  to  $n - 1$  do
3:     swapped  $\leftarrow$  false
4:     for  $j \leftarrow 0$  to  $n - 1 - i$  do
5:       if  $V[j] > V[j + 1]$  then
6:         swap( $V[j], V[j + 1]$ )
7:       swapped  $\leftarrow$  true
8:     if swapped = false then
9:       break

```

Se puede realizar el siguiente análisis de la ejecución del algoritmo.

Línea	Costo	Repeticiones en peor caso
2	c_1	n
3	c_2	n
4	c_3	$\sum_{i=1}^n (n - i) = n^2 - \frac{n^2 + n}{2} = \frac{n^2 - n}{2}$
5	c_4	$\sum_{i=1}^n (n - i) = n^2 - \frac{n^2 + n}{2} = \frac{n^2 - n}{2}$
6, 7	c_5	$\sum_{i=1}^n (n - i) = n^2 - \frac{n^2 + n}{2} = \frac{n^2 - n}{2}$
8	c_6	n
9	c_7	0

El **peor caso** se da cuando todas las líneas se ejecutan la máxima cantidad de veces (En este caso el arreglo nunca está ordenado por lo que nunca se ejecuta el `break` de la línea 9)

$$T(n) = (c_1 + c_2 + c_6)n + (c_3 + c_4 + c_5)\frac{n^2 - n}{2} \in O(n^2)$$

El **mejor caso** ocurre cuando el arreglo está ordenado lo que hace que el `break` se ejecute en la primera vuelta reduciendo los recorridos del arreglo a 1 solamente:

$$T(n) = c_1 + c_2 + (c_3 + c_4)n + c_6 + c_7 \in O(n)$$

Finalmente para el **caso promedio** se hace un análisis con la mitad de los intercambios, con esto tendremos:

$$T(n) = (c_1 + c_2 + c_6)n + (c_3 + c_4)\frac{n^2 - n}{2} + c_5\frac{n^2 - n}{4} \in O(n^2)$$

3.1.4. Ordenamiento por Inserción (Insertion sort)

Se puede entender este algoritmo como el método que usaría un jugador de cartas para organizar su mazo.

Este algoritmo divide el conjunto de datos en dos partes: la primera parte es el conjunto ordenado, y la segunda parte es el conjunto no ordenado. En cada paso, el i -ésimo elemento de la secuencia (el primero de la parte desordenada) se procesa y se transfiere a la secuencia destino, insertándolo en el lugar correspondiente.

El pseudocódigo asociado a dicho algoritmo es el siguiente:

Algorithm 4 Ordenacion por insercion $\triangleright V[0..n-1]$

```

1: procedure INSERTIONSORT( $V$ )
2:   for  $i \leftarrow 1$  to  $n-1$  do
3:      $k \leftarrow V[i]$  // Valor a ubicar
4:      $j \leftarrow i-1$  // Inicio de la búsqueda para ubicar
5:     while  $(j \geq 0) \wedge (V[j] > k)$  do
6:        $V[j+1] \leftarrow V[j]$ 
7:        $j \leftarrow j-1$ 
8:      $V[j+1] \leftarrow k$ 

```

Se puede realizar el siguiente análisis de la ejecución del algoritmo.

Línea	Costo	Repeticiones en peor caso
2	c_1	n
3, 4	c_2	n
5	c_3	$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$
6, 7	c_4	$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$
8	c_5	n

El **peor caso** se da cuando todas las líneas se ejecutan la máxima cantidad de veces:

$$T(n) = (c_1 + c_2 + c_5)n + (c_3 + c_4)\frac{n^2 + n}{2} \in O(n^2)$$

El **mejor caso** ocurre cuando el arreglo está ordenado, en este caso el while solamente hace una comparación por vez reduciendo el total de repeticiones a n y evitando la ejecución de lo que está en su interior.

$$T(n) = (c_1 + c_2 + c_4 + c_5)n \in O(n)$$

Finalmente para el **caso promedio** se hace un análisis con la mitad de las ejecuciones al interior del while:

$$T(n) = (c_1 + c_2 + c_5)n + c_3\frac{n^2 - n}{2} + c_4\frac{n^2 - n}{4} \in O(n^2)$$

3.1.5. Ordenamiento por selección (Selection sort)

El tercer algoritmo que se verá es el *Selection sort*.

Este método consiste en seleccionar el menor elemento del conjunto y a continuación intercambiarlo con el elemento que ocupa la primera posición del mismo. Hay que repetir esta operación con los $n-1$ elementos restantes, luego los $n-2$ elementos restantes y así sucesivamente hasta que solo quede un elemento.

Existe una optimización de este algoritmo que evita realizar operaciones de asignación innecesarias como si en un recorrido el elemento a intercambiar es el mismo que el menor elemento, en cuyo caso se omite el intercambio.

El pseudocódigo asociado a dicho algoritmo optimizado es el siguiente:

Algorithm 5 Ordenacion por seleccion ▷ $V[0..n-1]$

```

1: procedure SELECTIONSORT( $V$ )
2:   for  $i \leftarrow 1$  to  $n-1$  do
3:      $k \leftarrow i-1$  // Índice a desplazar
4:      $m \leftarrow V[i-1]$  // Inicialmente el menor
5:     for  $j \leftarrow i$  to  $n-1$  do
6:       if  $V[j] < m$  then
7:          $k \leftarrow j$ 
8:          $m \leftarrow V[j]$ 
9:     if  $k \neq i-1$  then
10:       $V[k] \leftarrow V[i-1]$ 
11:       $V[i-1] \leftarrow m$ 

```

Se puede realizar el siguiente análisis de la ejecución del algoritmo.

Línea	Costo	Repeticiones en peor caso
2	c_1	n
3, 4	c_2	n
5	c_3	$\sum_{i=1}^n (n-i) = n^2 - \frac{n^2+n}{2} = \frac{n^2-n}{2}$
6	c_4	$\sum_{i=1}^n (n-i) = n^2 - \frac{n^2+n}{2} = \frac{n^2-n}{2}$
7, 8	c_5	$\sum_{i=1}^n (n-i) = n^2 - \frac{n^2+n}{2} = \frac{n^2-n}{2}$
9	c_6	n
10, 11	c_7	n

El **peor caso** se da cuando todas las líneas se ejecutan la máxima cantidad de veces:

$$T(n) = (c_1 + c_2 + c_6 + c_7)n + (c_3 + c_4 + c_5)\frac{n^2 - n}{2} \in O(n^2)$$

El **mejor caso** ocurre cuando el arreglo está ordenado, en este caso las líneas 7, 8, 10, 11 no se ejecutan, sin embargo la complejidad temporal no disminuye.

$$T(n) = (c_1 + c_2 + c_6)n + (c_3 + c_4)\frac{n^2 - n}{2} \in O(n^2)$$

Finalmente para el **caso promedio** se hace un análisis con la mitad de las comparaciones verdaderas, con esto tendremos:

$$T(n) = (c_1 + c_2 + c_6)n + (c_3 + c_4)\frac{n^2 - n}{2} + (c_5 + c_7)\frac{n^2 - n}{4} \in O(n^2)$$

Note que a diferencia del *Swap sort* el *Selection sort* no tiene una mejora de complejidad temporal en el mejor caso a nivel de análisis asintótico.

3.1.6. Ordenamiento de coctelera (*Cocktail-Shaker Sort*)

Como menciona Knuth [2, § 5.2.2], una deficiencia del Swap Sort es que “los elementos nunca pueden moverse más de una posición a la izquierda por recorrido; por lo que si el elemento más pequeño se encuentra inicialmente en el extremo derecho, se está forzado a realizar la cantidad máxima de comparaciones”. Esto motiva una variante: “el *cocktail-shaker sort*, en el cual los recorridos alternan de dirección” [2, § 5.2.2], reduciendo así el número promedio de comparaciones respecto al algoritmo original.

El pseudocódigo asociado a dicho algoritmo es el siguiente:

Algorithm 6 Cocktail Shaker Sort ▷ $V[0..n-1]$

```

1: procedure COCKTAILSHAKERSORT( $V$ )
2:    $beg \leftarrow 0$ 
3:    $end \leftarrow n - 1$ 
4:   while  $beg < end$  do
5:      $swapped \leftarrow 0$ 
6:     for  $i \leftarrow beg$  to  $end - 1$  do
7:       if  $V[i] > V[i + 1]$  then
8:          $swap(V[i], V[i + 1])$ 
9:          $swapped \leftarrow 1$ 
10:    for  $i \leftarrow end - 1$  downto  $beg + 1$  do
11:      if  $V[i] < V[i - 1]$  then
12:         $swap(V[i], V[i - 1])$ 
13:         $swapped \leftarrow 1$ 
14:    if  $swapped = 0$  then
15:      break
16:     $beg \leftarrow beg + 1$ 
17:     $end \leftarrow end - 1$ 

```

Se puede realizar el siguiente análisis de la ejecución del algoritmo.

Línea	Costo	Repeticiones en peor caso
2, 3	c_1	1
4	c_2	$n/2$
5	c_3	$n/2$
6	c_4	$\sum_{i=1}^{n/2} (n - 2i) = \frac{n^2}{2} - 2 \left(\frac{\frac{n}{2}(\frac{n}{2} + 1)}{2} \right) = \frac{n^2}{2} - \frac{n^2}{4} - \frac{n}{2} = \frac{n^2 - 2n}{4}$
7	c_5	$\frac{n^2 - 2n}{4}$
8, 9	c_6	$\frac{n^2 - 2n}{4}$
10	c_7	$\frac{n^2 - 2n}{4}$
11	c_8	$\frac{n^2 - 2n}{4}$
12, 13	c_9	$\frac{n^2 - 2n}{4}$
14	c_{10}	$n/2$
15	c_{11}	0
16, 17	c_{12}	$n/2$

El **peor caso** se da cuando todas las líneas se ejecutan la máxima cantidad de veces:

$$T(n) = c_1 + (c_2 + c_3 + c_9 + c_{11})\frac{n}{2} + (c_4 + c_5 + c_6 + c_7 + c_8 + c_9)\frac{n^2 - 2n}{4} \in O(n^2)$$

El **mejor caso** ocurre cuando el arreglo está ordenado lo que hace que el `break` se ejecute en la primera vuelta reduciendo los recorridos del arreglo a 1 solamente:

$$T(n) = c_1 + c_2 + c_3 + (c_4 + c_5 + c_7 + c_8)(n - 2) + c_{10} + c_{11} \in O(n)$$

Finalmente para el **caso promedio** se hace un análisis con la mitad de los intercambios, con esto tendremos:

$$T(n) = c_1 + (c_2 + c_3 + c_{10} + c_{12})\frac{n}{2} + (c_4 + c_5 + c_7 + c_8)\frac{n^2 - 2n}{4} + (c_6 + c_9)\frac{n^2 - 2n}{8} \in O(n^2)$$

Note que a nivel asintótico la complejidad temporal de esta variación del *Swap sort* se mantiene, sin embargo el término más relevante es $2 \cdot \frac{n^2 - 2n}{4}$ (Por los dos ciclos For presentes) en vez de $\frac{n^2 - n}{2}$ lo que se traduce en un menor tiempo de ejecución.

3.2. Implementación de Algoritmos de Ordenamiento

A pesar de tener un conocimiento teórico de los algoritmos a implementar (3.1) durante el desarrollo de la solución, surgió la necesidad de ordenar estructuras de datos complejas (los registros correspondientes a los deportistas (`Player`) en lugar de tipos de datos primitivos como números enteros.

Para abordar esta problemática, se tomó como base el diseño de la función de la biblioteca estándar de C, `qsort()` [3]. Esta función resuelve el problema de la genericidad mediante el uso de funciones como parámetro, delegando el criterio de ordenación a una función externa. Adoptando este mismo patrón, se modificaron los algoritmos de ordenamiento. Se puede ver un ejemplo de cómo se implementó el uso de funciones como parámetro en los bloques de código 2 y 3.

El parámetro `int (*comp_f)(Player *, Player *)` permite desacoplar la lógica interna del algoritmo de ordenamiento del atributo específico que se desea comparar. De este modo, el mismo algoritmo es capaz de ordenar el arreglo por identificador, nombre, equipo, puntuación o cantidad de competencias, dependiendo de la función de comparación que se le inyecte en tiempo de ejecución (Bloque de código 1).

```
1 int compare_id(Player *p1, Player *p2);
2 int compare_name(Player *p1, Player *p2);
3 int compare_team(Player *p1, Player *p2);
4 int compare_score(Player *p1, Player *p2);
5 int compare_competitions(Player *p1, Player *p2);
```

Código 1. Prototipos de las funciones de comparación en `player.h`

Revisión de las funciones de comparación de cadenas

En la fase inicial de pruebas, se detectó un comportamiento anómalo al realizar el ordenamiento alfabético por nombre y equipo. Al implementar la función estándar `strcmp()` (De la librería `<string.h>`), el ordenamiento resultante priorizaba las letras mayúsculas sobre las minúsculas. Esto se debe a que `strcmp()` realiza una comparación estricta basada en los valores decimales de la tabla ASCII, donde, por ejemplo, el carácter 'Z' (valor 90) se considera estrictamente menor que el carácter 'a' (valor 97).

Como resultado de este comportamiento, el arreglo ordenado presentaba inconsistencias lexicográficas, agrupando primero todas las cadenas iniciadas con mayúsculas:

- **Resultado previo (sensible a mayúsculas):** `["Alvarez", "Zarate", "aguilar", "bernal"]`

Para corregir este defecto y garantizar un ordenamiento natural e intuitivo, se procedió a refactorizar las funciones `compare_name()` y `compare_team()`. Se sustituyó la función `strcmp()` por `strcasecmp()` de la misma librería. Esta última función realiza la resta de los caracteres ignorando la diferencia entre mayúsculas y minúsculas (convirtiendo internamente los caracteres a un formato uniforme antes de la comparación).

Con esta implementación, se logró unificar el criterio de orden alfabético, obteniendo resultados insensibles a capitalización:

- **Resultado final (insensible a mayúsculas):** `["aguilar", "Alvarez", "bernal", "Zarate"]`

3.3. Generador de deportistas

El generador de deportistas fue implementado con el objetivo de producir conjuntos de datos artificiales que sirvan como entrada para los algoritmos de ordenamiento y búsqueda desarrollados en el proyecto. Su funcionamiento se basa en la creación dinámica de un arreglo de

estructuras `Player`. Para cada registro se genera un identificador único autoincremental, un nombre aleatorio compuesto por letras mayúsculas y minúsculas, un equipo seleccionado aleatoriamente desde un conjunto predefinido, un puntaje decimal y una cantidad de competencias. Además, el generador contempla tres modalidades de salida: arreglo ordenado, donde los registros se mantienen en el orden original de generación (con identificadores de menor a mayor); arreglo invertido, donde el arreglo completo es invertido según identificadores; y arreglo desordenado, donde se aplica una permutación aleatoria mediante el algoritmo de Fisher-Yates [1, § 3.4.2].

En términos de almacenamiento, los deportistas se generan primero en memoria dinámica utilizando `malloc()`, lo que permite trabajar con tamaños de entrada variables y acordes al experimento. Una vez completada la generación y, si corresponde, el reordenamiento del arreglo, los datos se escriben en el archivo `build/db/players.csv`. En la versión actual, los campos se separan mediante espacios en lugar de comas, decisión adoptada para facilitar la carga posterior mediante `fscanf()`.

El proceso de lectura fue implementado en la función `load_players()`, cuya tarea es reconstruir en memoria el arreglo de deportistas a partir del archivo previamente generado.

3.4. Verificación Experimental de la Complejidad Temporal

Para verificar empíricamente la complejidad temporal de los algoritmos implementados, se diseñó la función `run_experiment()`, que mide los tiempos de ejecución de cada algoritmo sobre conjuntos de datos de tamaño variable y registra los resultados en un archivo CSV.

El experimento subdivide el rango $[1, n]$ en `NUM_STEPS` puntos equiespaciados. Para cada paso s , el tamaño efectivo k se calcula como:

$$k = 1 + \frac{s \cdot (n - 1)}{\text{NUM_STEPS} - 1}, \quad s = 0, 1, \dots, \text{NUM_STEPS} - 1$$

Esto garantiza que el primer punto corresponda a $k = 1$, el último a $k = n$, y que los puntos intermedios estén distribuidos de forma uniforme, obteniendo así una curva representativa del comportamiento de cada algoritmo a lo largo de distintos tamaños de entrada.

Para cada valor de k y cada algoritmo se realizaron `NUM_TRIALS` repeticiones. En cada repetición se siguió el siguiente procedimiento:

1. Se copió el subarreglo original de tamaño k en un arreglo auxiliar mediante `memcpy()`, garantizando que cada ejecución parta desde el mismo estado inicial.
2. Se registró el instante anterior y posterior a la llamada al algoritmo utilizando `clock()`.
3. Se acumuló el tiempo transcurrido para esa repetición.

El tiempo reportado para cada par (*algoritmo*, k) corresponde al **promedio** de las `NUM_TRIALS` repeticiones, reduciendo el efecto de variaciones introducidas por el comportamiento del sistema operativo.

Consideración para la búsqueda binaria

Dado que la búsqueda binaria requiere como precondition un arreglo ordenado, se tomó como entrada el arreglo ya procesado por uno de los algoritmos de ordenamiento en la misma iteración, en lugar del arreglo original desordenado. Esto asegura una medición válida del algoritmo sin incorporar el costo del ordenamiento previo en su tiempo registrado.

4. Resultados experimentados

Todos los experimentos se realizaron considerando un conjunto de **7000 elementos** (deportistas), evaluando el comportamiento de los algoritmos en 20 pasos equidistantes. Para mitigar el impacto de variaciones en el rendimiento del procesador, cada experimentación se ejecutó **1000 veces**, tomando promedios de ejecución como se describió en la sección 3.4.

4.1. Algoritmos de Búsqueda

Para evaluar los algoritmos de búsqueda se seleccionó un **elemento con identificador $n + 1$ en un arreglo de tamaño n** . Esta estrategia garantiza que el elemento buscado nunca existe en la estructura de datos, asegurando que ambos algoritmos siempre enfrenten su **peor caso posible**: la búsqueda exhaustiva sin encontrar el elemento objetivo.

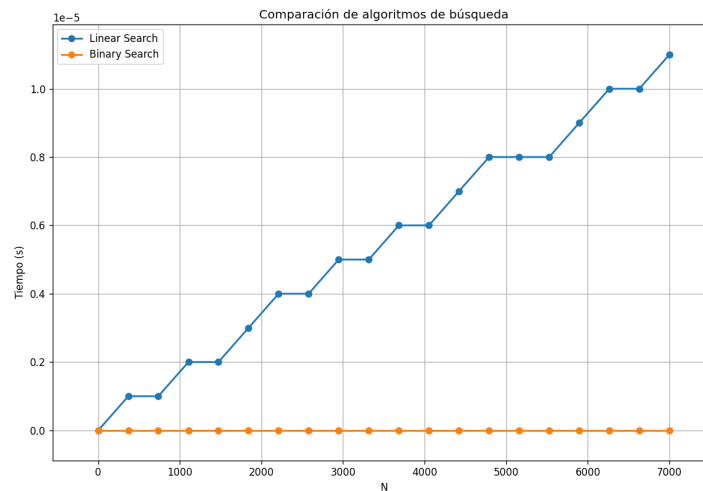


Figura 1. Comparación de algoritmos de búsqueda partiendo de un arreglo desordenado

Los resultados demuestran una diferencia radical de rendimiento entre los dos algoritmos. Es importante destacar que **Binary Search** requiere un arreglo previamente ordenado; sin embargo, el tiempo de ordenamiento **no está contabilizado** en el tiempo de ejecución del algoritmo de búsqueda.

Análisis comparativo:

- **Linear Search** muestra un crecimiento lineal en el tiempo de ejecución, aumentando proporcionalmente con el tamaño del arreglo y alcanzando tiempos significativos en el peor caso.
- **Binary Search** mantiene un tiempo de ejecución prácticamente constante y cercano a cero en toda la escala, demostrando que su complejidad es más eficiente que Linear Search.

Conclusión: Binary Search es significativamente más eficiente. Teóricamente, con un arreglo de 7000 elementos, Binary Search requiere como máximo $\lceil \log_2(7000) \rceil = 13$ comparaciones, ya que $2^{13} = 8192 > 7000$. En contraste, Linear Search debe realizar iteraciones que pueden alcanzar los 7000 intentos en el peor caso. Esta ventaja tan abismal es la que genera el comportamiento de aspecto $O(1)$ en el comportamiento de Binary Search.

4.2. Algoritmos de Ordenamiento

Los algoritmos de ordenamiento se evaluaron bajo **tres escenarios distintos** para capturar comportamientos variados:

4.2.1. Arreglo Ordenado (Mejor Caso)

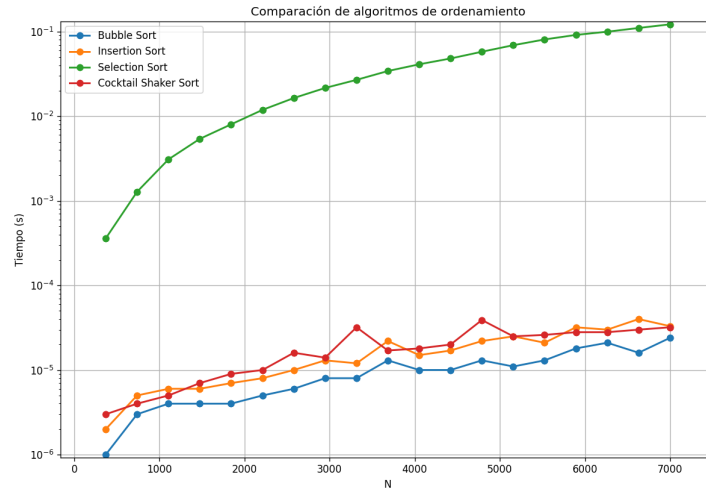


Figura 2. Comparación de algoritmos de ordenamiento con arreglo ya ordenado (escala logarítmica)

En el mejor caso, donde el arreglo ya se encuentra ordenado, se observa claramente que **Selection Sort** es el algoritmo más desventajoso. A diferencia de otros algoritmos que pueden reconocer el estado ordenado y terminar anticipadamente, Selection Sort realiza comparaciones exhaustivas que nunca disminuyen, independientemente del estado inicial de los datos. Su complejidad $O(n^2)$ permanece constante incluso cuando es innecesaria toda la labor de comparación.

4.2.2. Arreglo en Orden Inverso (Peor Caso)

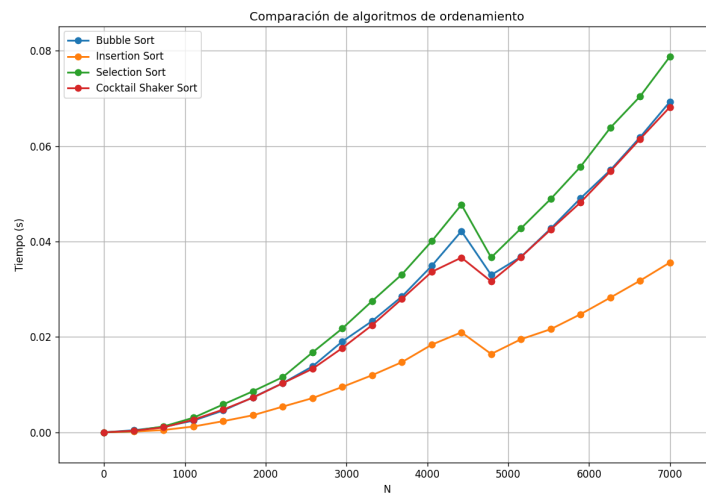


Figura 3. Comparación de algoritmos de ordenamiento con arreglo en orden inverso

En el peor caso (arreglo en orden inverso), los resultados revelan que **Insertion Sort** es considerablemente más eficiente que sus competidores, ejecutándose en aproximadamente la mitad del tiempo de Selection Sort.

4.2.3. Arreglo Desordenado (Caso Promedio)

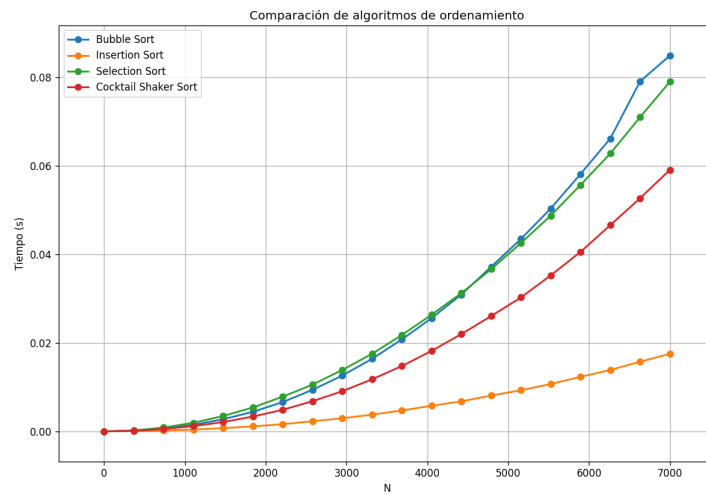


Figura 4. Comparación de algoritmos de ordenamiento con arreglo en desorden

En el caso promedio, con datos aleatorios, **Insertion Sort** se posiciona nuevamente como el algoritmo más eficiente. Selection Sort continúa mostrando bajo rendimiento, siendo superado en los últimos puntos por Swap Sort, aunque la diferencia es marginal.

4.2.4. Comportamiento de Cocktail Shaker Sort:

Cocktail Shaker Sort (3.1.6) es una optimización de Swap Sort que realiza búsquedas bidireccionales (ida y vuelta). Su desempeño relativo varía según el escenario:

- **Mejor caso:** Swap Sort es más eficiente porque Cocktail Shaker Sort realiza una pasada completa en ambas direcciones antes de detectar que el arreglo está ordenado.
- **Peor caso:** Ambos algoritmos exhiben comportamiento prácticamente idéntico, reflejando sus complejidades $O(n^2)$.
- **Caso promedio:** Cocktail Shaker Sort demuestra ser **considerablemente más eficiente** que Swap Sort estándar, aprovechando su capacidad de detectar patrones en ambas direcciones.

5. Conclusiones

A partir del análisis teórico y experimental realizado, se extraen las siguientes conclusiones respecto al comportamiento de los algoritmos estudiados:

Eficiencia comparativa de algoritmos de búsqueda

La Búsqueda Binaria demostró ser significativamente más eficiente que la Búsqueda Secuencial, con una complejidad temporal de $\mathcal{O}(\log n)$ frente a $\mathcal{O}(n)$. Teóricamente, sobre un arreglo de 7000 elementos, Binary Search requiere como máximo $\lceil \log_2(7000) \rceil = 13$ comparaciones, mientras que Linear Search puede alcanzar 7000 intentos en el peor caso. Si bien la velocidad de procesamiento de los computadores actuales hace que las diferencias en órdenes logarítmicos sean difíciles de capturar en tiempos absolutos (frecuentemente apareciendo como valores cercanos a cero), el análisis experimental permitió evidenciar la mejora relativa de la Búsqueda Binaria, reflejando las predicciones teóricas sobre su superioridad asintótica.

Eficiencia comparativa de algoritmos de ordenamiento

Los resultados experimentales revelaron distintos niveles de eficiencia entre los algoritmos de ordenamiento evaluados:

1. **Algoritmo más eficiente:** Insertion Sort demostró el mejor desempeño en todos los escenarios evaluados (mejor caso, peor caso y caso promedio), ejecutándose aproximadamente en la mitad del tiempo requerido por Selection Sort en el peor caso.
2. **Algoritmo menos eficiente:** Selection Sort presentó el comportamiento más desfavorable, especialmente en el mejor caso, donde su complejidad $\mathcal{O}(n^2)$ permanece constante incluso cuando el arreglo ya se encuentra ordenado, a diferencia de otros algoritmos que pueden terminar anticipadamente.
3. **Swap Sort y su variante:** Aunque ambos exhiben complejidad $\mathcal{O}(n^2)$, Cocktail-Shaker Sort (variante bidireccional de Swap Sort) demostró ser considerablemente más eficiente que el Swap Sort estándar en el caso promedio, aprovechando su capacidad de detectar patrones de orden en ambas direcciones. Sin embargo, en el mejor caso (arreglo ya ordenado) Swap Sort resulta superior, mientras que en el peor caso ambos exhiben comportamiento prácticamente idéntico.

Validación de la teoría en la práctica

El análisis experimental corroboró las predicciones del análisis teórico de complejidad. Las curvas de tiempo de ejecución se ajustaron a los órdenes asintóticos predichos, confirmando que el análisis de complejidad constituye una herramienta confiable para la predicción del comportamiento de algoritmos.

Implicaciones para la selección de algoritmos

Aunque los algoritmos analizados representan casos de estudio académicos con complejidades cuadráticas o lineales, los hallazgos refuerzan la importancia de considerar la complejidad temporal al seleccionar algoritmos. La diferencia relativa entre algoritmos de similar orden de complejidad (como Swap Sort e Insertion Sort) puede ser sustancial en aplicaciones prácticas con grandes volúmenes de datos. Asimismo, la factibilidad de optimizaciones (como Cocktail-Shaker Sort) demuestra que mejoras significativas pueden obtenerse mediante modificaciones algorítmicas sin cambiar el orden de complejidad.

6. Contribución por integrante

A continuación se detalla la contribución realizada por cada uno de los integrantes del grupo en el desarrollo del proyecto.

6.1. Andrés Barbosa

El estudiante **Andrés Barbosa** contribuyó al proyecto con las siguientes tareas:

- Implementación de los algoritmos de Búsqueda y Ordenamiento sobre estructuras `Player`.
- Redacción de la sección 3.2.
- Creación de las diapositivas para la precondition del proyecto.

6.2. Milton Hernández

El estudiante **Milton Hernández** contribuyó al proyecto con las siguientes tareas:

- Coordinación del equipo.
- Organización de la estructura de archivos del proyecto, normas de codificación y de trabajo.
- Redacción de las secciones: 3.1, 3.4 y 4.
- Implementación del experimento de ordenamiento y búsqueda.

6.3. Iván Gallardo

El estudiante **Iván Gallardo** contribuyó al proyecto con las siguientes tareas:

- Implementación inicial de algoritmos de Búsqueda y Ordenamiento.
- Creación del generador de deportistas.
- Creación y estructura del menú de opciones del programa.
- Confección de la funcionalidad de graficación mediante `bash` y `Python`.
- Creación de la CLI del programa.
- Redacción de la sección 3.3.

7. anexos

```

1  /**
2   * @brief Ordenamiento por intercambio
3   *
4   * @param V arreglo de enteros
5   * @param n tamanho del arreglo
6   */
7  void swap_sort(int V[], int n)
8  {
9      for (int i = 1; i <= n - 1; i++) {
10         int swapped = 0;
11
12         for (int j = 0; j <= n - 1 - i; j++) {
13             if (V[j] > V[j + 1]) {
14                 swap(&V[j], &V[j + 1]);
15                 swapped = 1;
16             }
17         }
18
19         if (swapped == 0) {
20             printf("There was no swapping\n");
21             break;
22         }
23     }
24 }

```

Código 2. Implementación de SwapSort para números enteros

```

1  /**
2   * @brief Ordenamiento por intercambio
3   *
4   * @param V arreglo de enteros
5   * @param n tamanho del arreglo
6   * @param comp_f funcion de comparacion (1 si a < b, -1 si a > b, 0 si a = b)
7   */
8  void swap_sort(Player V[], int n, int(*comp_f)( Player *, Player *))
9  {
10     for (int i = 1; i <= n - 1; i++) {
11         int swapped = 0;
12
13         for (int j = 0; j <= n - 1 - i; j++) {
14             if (comp_f(&V[j], &V[j + 1]) > 0) {
15                 swap(&V[j], &V[j + 1]);
16                 swapped = 1;
17             }
18         }
19
20         if (swapped == 0) {
21             break;
22         }
23     }
24 }

```

Código 3. Implementación de SwapSort para jugadores (estructuras Player)

■ Referencias

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical algorithms*, 3.^a ed. Addison-Wesley, 1997, ISBN: 978-0-201-89684-8. visitado 13 de nov. de 2019.
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2.^a ed. Addison-Wesley, 1998, ISBN: 978-0-201-89685-5. visitado 2 de abr. de 2026.
- [3] Linux Kernel Organization, *Linux Programmer's Manual: QSORT(3)*, Accedido el 12 de abril de 2026, 2024. dirección: <https://man7.org/linux/man-pages/man3/qsort.3.html>.